

Sebastian Vintoñuke – 106063

Cindy Teresa Hsieh – 108051

Maria Delfina Cano Ros Langrehr – 109338

Lautaro Martin Sotelo – 107472

Toda afirmación que se realice debe estar apoyada en **evidencia**. Si se encuentra alguna **afirmación** que no está correctamente fundamentada con **evidencia**, se disminuirá la nota. Esto incluye las observaciones que realicen sobre el código y su funcionamiento en general. Que no existan métricas para un determinado QA no implica que no pueda evidenciarse el efecto de una táctica.

1. SISTEMA RECIBIDO

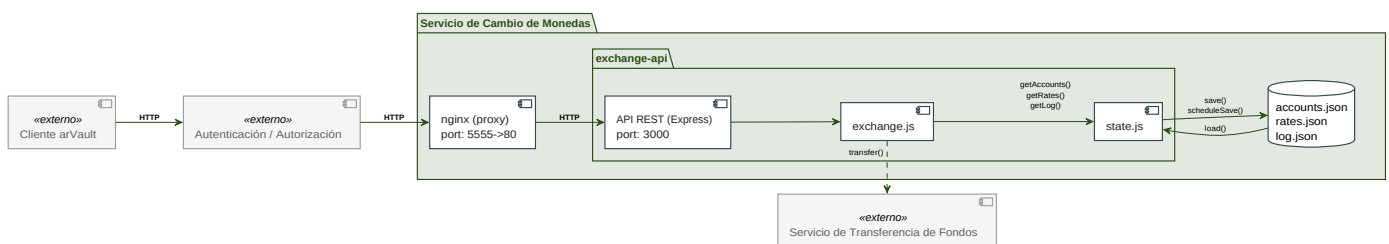


Figura 1. Diagrama Components & Connectors del sistema base.

1.1. Arquitectura

Arquitectura monolítica cliente-servidor, formada por una única instancia app.js. Separación en tres capas: app.js punto de entrada y definición de endpoints, exchange.js lógica de negocio, state.js persistencia. No se implementa coordinación, no hay locks ni mecanismos de concurrencia sobre secciones críticas, como la verificación del saldo. No se implementa tolerancia a fallos, no hay manejo de errores try/catch. No hay replicación, única instancia, no hay réplicas. No hay recovery, si el servidor muere hay que levantarlo manualmente. No hay consistencia entre los datos en memoria y los datos en disco, la consistencia es eventual sin mecanismos de transaccionabilidad.

TODO: redactarlo mas bonito y justificar

1.2. Código

No hay separación de responsabilidades, exchange maneja tanto lógica de negocio, como lógica de bajo nivel. Calcula tanto counterAmount como se encarga de persistir los cambios en counterAccount.balance.

```

1 export async function exchange(exchangeRequest) {
2   ...
3   //get the exchange rate
4   const exchangeRate = rates[baseCurrency][counterCurrency];
5   //compute the requested (counter) amount
6   const counterAmount = baseAmount * exchangeRate;
7   ...
8   //all good, update balances
9   baseAccount.balance += baseAmount;
10  counterAccount.balance -= counterAmount;
11  ...
12 }

```

Código 1. app/exchange.js

Hay alto acoplamiento, exchange.js accede directamente a variables que deberían ser privadas de state.js. exchange.js no debería conocer detalles de implementación de la persistencia, se debería programar contra interfaces. Cambiar la persistencia implica modificar exchange.js. No hay modelo de clases, todo está basado en constantes globales y funciones públicas.

```

1 let accounts;
2 let rates;
3 let log;
4
5 export async function init() {
6   await stateInit();
7 }

```

```

8   accounts = stateAccounts();
9   rates = stateRates();
10  log = stateLog();
11 }

```

Código 2. app/exchange.js

No se implementa transaccionalidad, posibles inconsistencias ante fallos incluso al recuperarse.

```

1  async function save(data, fileName) {
2    const filePath = path.join(__dirname, fileName);
3    try {
4      await fs.promises.writeFile(filePath, JSON.stringify(data, null, 2));
5    } catch (err) {
6      console.error(`Error writing to ${filePath}:`, err);
7    }
8  }

```

Código 3. app/state.js

No se implemento un diseño guiado por dominio, se opera directamente contra los tipos de datos primitivos. Se modela el dinero utilizando floats, esto genera perdida de precision.

```

1  const exchangeRate = rates[baseCurrency][counterCurrency];
2  //compute the requested (counter) amount
3  const counterAmount = baseAmount * exchangeRate;

```

Código 4. app/exchange.js

1.3. Infraestructura

2. ATRIBUTOS DE CALIDAD

2.1. Identificación de los QA claves

Se identifican los siguientes atributos de calidad como claves en el contexto del negocio. Adicionalmente, se citan especificaciones y requerimientos que motivaron estas afirmaciones.

2.1.1. Availability (disponibilidad)

Habilidad del sistema para reparar o enmascarar fallas de manera tal que el período fuera de servicio no exceda un determinado valor durante un lapso de tiempo especificado.

La startup arVault es una fintech que opera una billetera digital

Al ser una billetera digital, el servicio permite manipular el dinero de los usuarios. Un servicio con baja o nula disponibilidad implicaría que los usuarios no pueden acceder a su propio dinero. Adicionalmente, el modelo de negocio se basa en que los usuarios operen, si el servicio no es disponible para permitir estas operaciones, se estan perdiendo potenciales ganancias.

2.1.2. Performance

Habilidad del sistema para reaccionar ante ciertos eventos en un determinado tiempo.

Si bien el lanzamiento fue exitoso, comenzaron a aparecer reclamos de los usuarios sobre problemas en el uso de la función de cambio de monedas.

Uno de los principales problemas que impulsaron esta review es el endpoint de /exchanges, como se mostro es (TODO) este considerablemente mas lento que el resto de endpoints, lo que podria implicar el descontento de los usuarios.

las tasas de cambio no tienen gap entre el valor de compra y de venta.

Adicionalmente, el modelo de negocios es ofrecer una tasa de cambio sin gap, es decir en tiempo real. Una performance pobre en esta operacion implicaría que las tasas podrian que tener que ajustarse negativamente en comparacion a lo que se le ofrece al usuario en primera instancia.

2.1.3. Scalability (escalabilidad)

Habilidad del sistema para incorporar recursos adicionales de manera efectiva. Efectividad = mejora mensurable de algún QA + no requerir un esfuerzo inapropiado + no interrumpir el funcionamiento.

Para ganar usuarios, las tasas de cambio...

Se especifica que objetivo a corto plaso es incrementar el numero de usuarios, el cual, el sistema debe ser capaz de manejar incorporando los recursos necesarios.

2.1.4. Reliability (confiabilidad)

Grado en que la arquitectura es susceptible a fallas a nivel de sistema cuando se producen fallas parciales en componentes, conectores o datos.

TODO: TODO: justificar

2.1.5. Security (seguridad)

Capacidad del sistema de prevenir acciones maliciosas o accidentales fuera del uso esperado.

se valida que haya saldo en la cuenta del cliente, se espera que lo haga la UI

La posibilidad de que los usuarios puedan realizar acciones maliciosas implican perdidas monetarias para el resto de usuarios, para la propia organizacion, ademas de una perdida de reputacion y con esto perdida de usuarios e inversiones.

2.2. Otros QA menos relevantes

Ademas se mencionan otros QA, que aunque no resulten claves y por ende no son una prioridad, son desceables, especialmente conforme el proyecto crezca.

TODO: TODO: busca en el enunciado o en la documentacion como justificar estos QA

2.2.1. Efficiency

Performance por recurso consumido.

2.2.2. Testability (testeabilidad)

Grado en que un componente/sistema de software permite ser probado para exhibir sus fallas; usualmente relacionado al esfuerzo necesario para llevar adelante las tareas de testing.

2.2.3. Manageability

Define la facilidad de gestión del sistema por parte de los administradores del mismo.

2.2.4. Simplicity

Facilidad con la que la arquitectura de un sistema puede ser entendida, razonada y explicada.

2.3. Decisiones de diseño actuales y su impacto en los QA**2.3.1. Impacto en la Performance**

...

2.4. Critica**3. COMPORTAMIENTO DEL SERVICIO****3.1. Pruebas de carga****3.2. Metricas****3.2.1. Latencia**

...

4. MODIFICACIONES PROPUESTAS

4.1. Propuesta 1

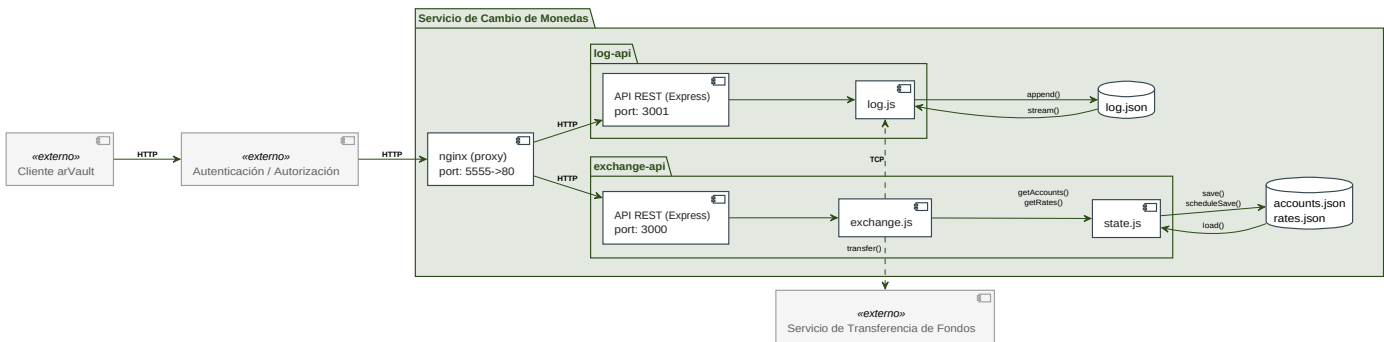


Figura 2. Diagrama Components & Connectors de la propuesta 1.

4.2. Impacto en los QA

4.2.1. Performance

...

4.3. Pruebas de carga

4.4. Métricas

4.4.1. Latencia

...

4.5. Resultados y conclusiones

5. MÉTRICAS ADICIONALES

5.1. Volumen operado por moneda

5.2. Volumen neto operado

6. USO DE HERRAMIENTAS DE IA

...

A. APENDICE

A.1. Resultados completos del escenario plano sobre el sistema base

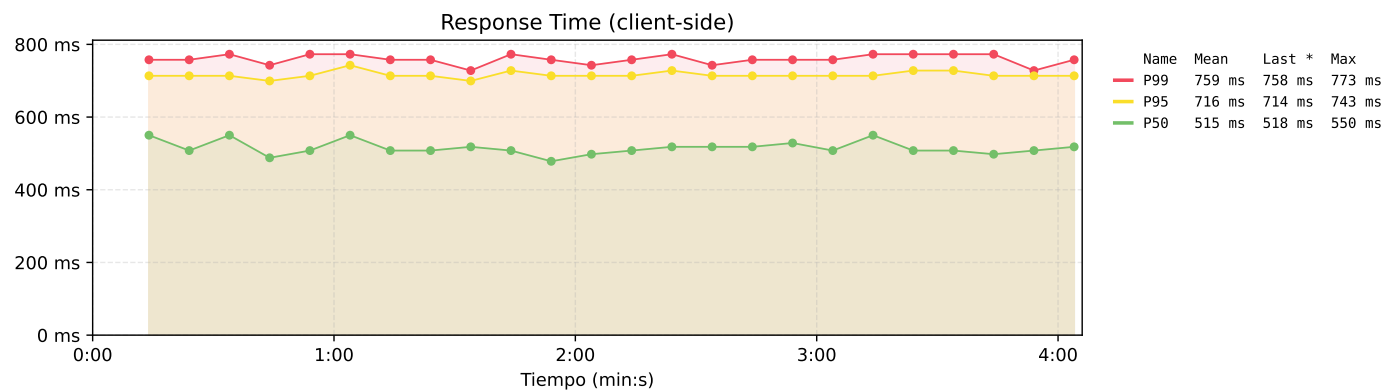


Figura 3. Tiempo de respuesta del sistema base.

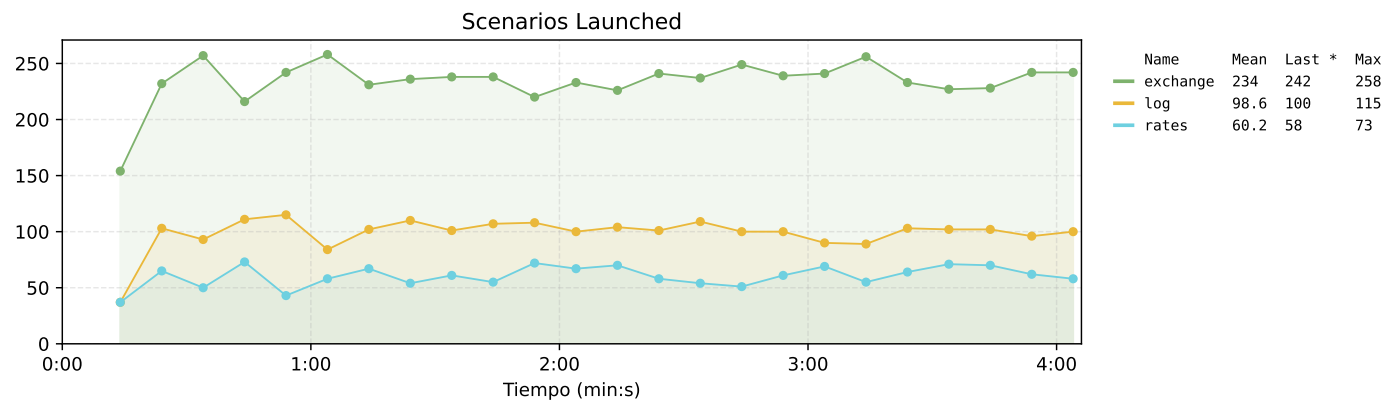


Figura 4. Escenarios del sistema base.

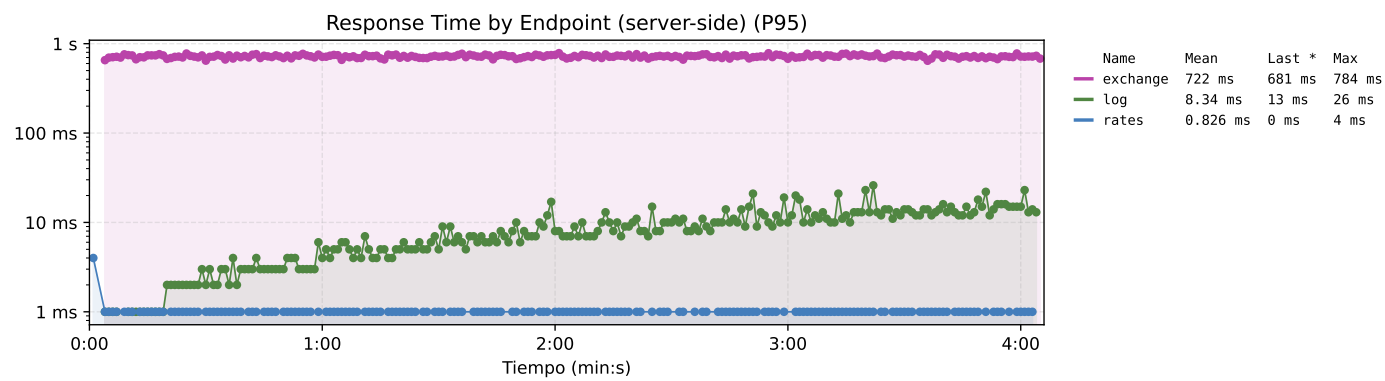


Figura 5. Tiempo de respuesta por endpoint del sistema base.

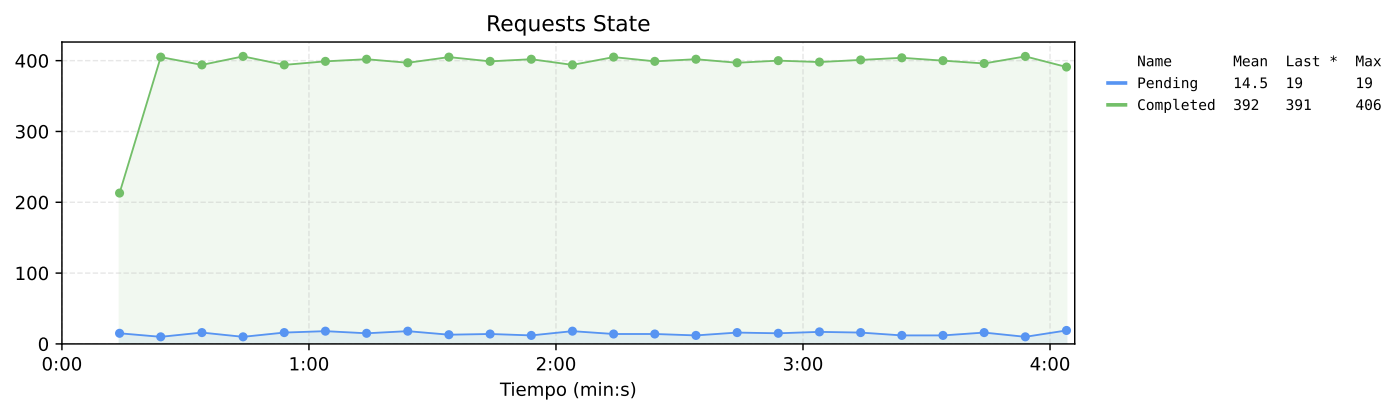


Figura 6. Estado de las requests del sistema base.

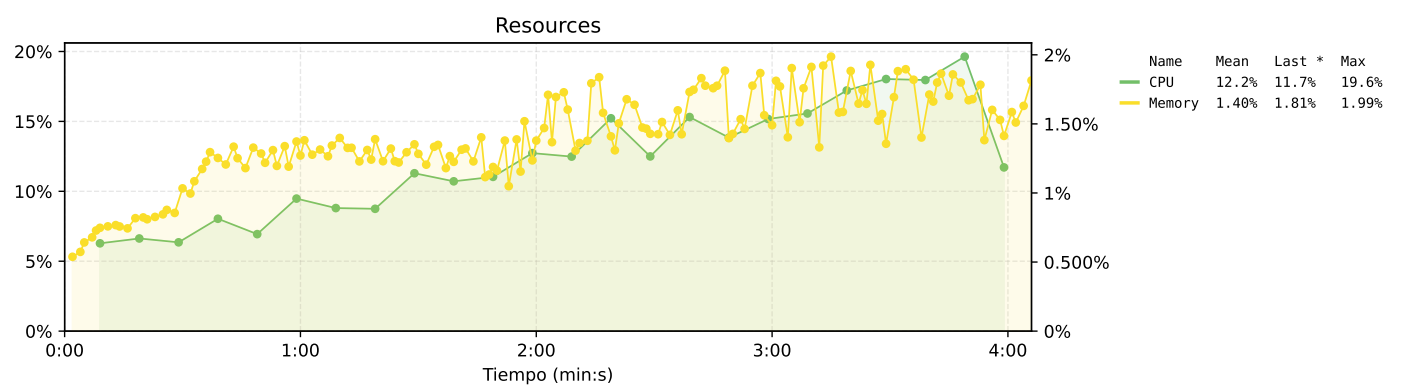


Figura 7. Estado de los recursos del sistema base.

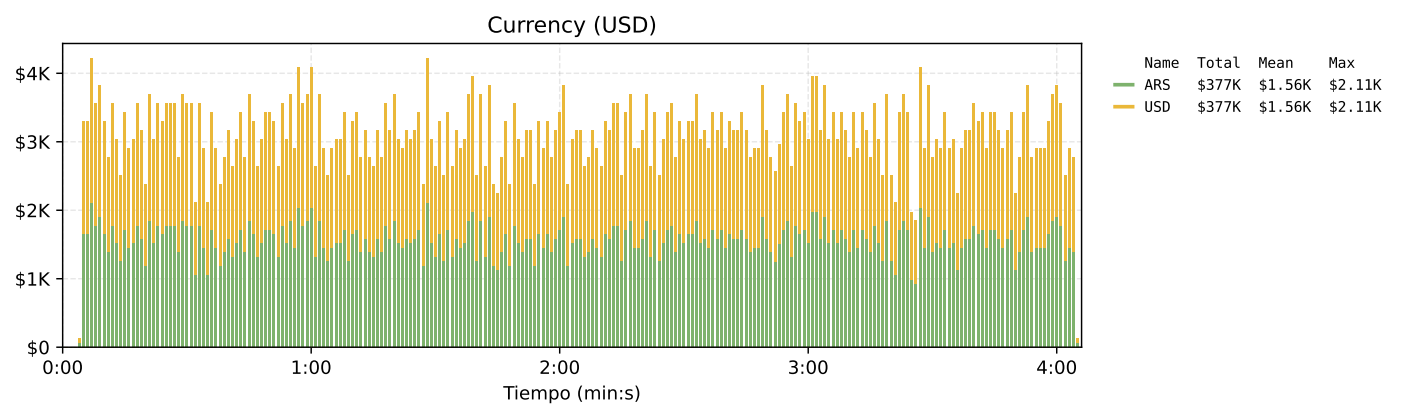


Figura 8. Volumen operado en cada moneda en el sistema base.

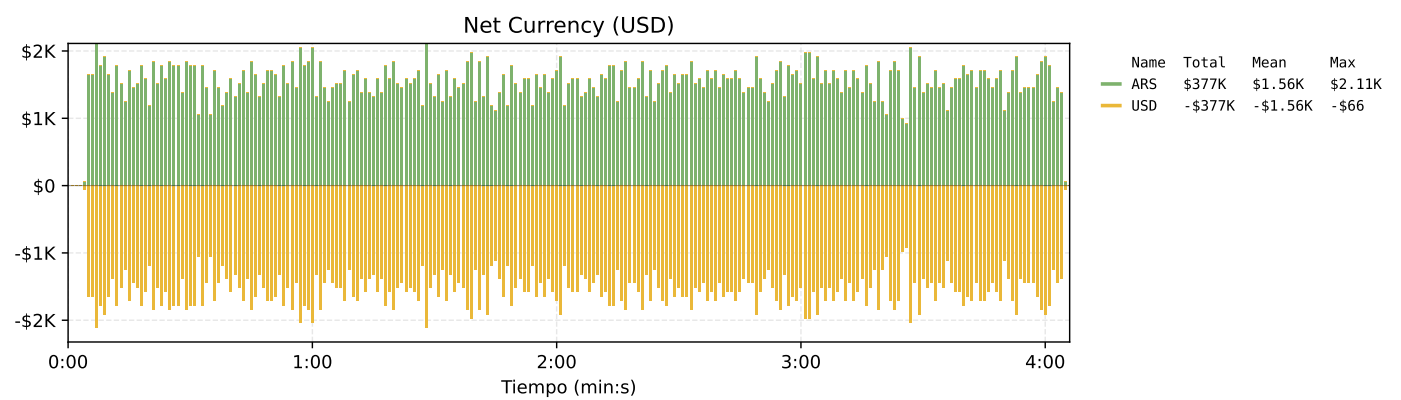


Figura 9. Neto operado en cada moneda en el sistema base.